

Implementação Exclusão (Autor)

Para excluir um autor é fundamental evitar que o banco de dados fique com “registros órfãos” (ou seja, livros que apontam para um autor que não existe mais)

Para resolver isso é necessário fazer uma verificação lógica em duas etapas:

- **No GET:** e o autor tiver livros, a tela de confirmação exibe um alerta vermelho e esconde/desativa o botão de excluir.
- **No POST:** Por segurança (caso alguém tente burlar o HTML), a Controller barra a exclusão se houver livros vinculados.

Vamos ao passo a passo de um exemplo de implementação:

1º Criar a ViewModel de Exclusão do Autor

Necessário criar o arquivo `Models/DeletarAutorViewModel.cs`. Esta classe transportará o nome do autor e uma propriedade booleana `PossuiLivrosVinculados` que controlará o comportamento da tela.

```
namespace Biblioteca.Models;

public class DeletarAutorViewModel
{
    public int Id { get; set; }
    public string? Nome { get; set; }
    public bool PossuiLivrosVinculados { get; set; } // Propriedad
e chave para a regra de negócio
}
```

2º Atualizar a Interface do Repositório de Autores

Alterar o arquivo `Repositories/AutorRepository.cs`. Precisamos de dois métodos: um para verificar se o autor possui livros e outro para deletar de fato. Então adicionamos as assinaturas deles:

```
using Biblioteca.Models;

namespace Biblioteca.Repositories;
```

```
public interface IAutorRepository
{
    Task<List<Autor>> BuscarTodosAutoresAsync();
    Task<bool> CriarAutorAsync(Autor autor);
    Task<bool> AtualizarAutorAsync(Autor autor);
    Task<bool> PossuiLivrosVinculadosAsync(int autorId); // → Adicionar esta linha
    Task<bool> ExcluirAutorAsync(int id); // → Adicionar esta linha
}
```

Ainda no arquivo `AutorRepository.cs` é necessário implementar a regra de verificação e exclusão do Repositório. O Entity Framework permite usar o método `.AnyAsync()` na tabela de Livros para checar de forma ultra rápida se existe qualquer livro associado àquele `Autor.Id`

```
// Certifique-se de ter o using do EF Core no topo do arquivo:
// using Microsoft.EntityFrameworkCore;

public async Task<bool> PossuiLivrosVinculadosAsync(int autorId)
{
    // Varre a tabela de Livros procurando se algum possui o ID do autor fornecido
    return await _context.Livros.AnyAsync(l => l.Autor.Id == autorId);
}

public async Task<bool> ExcluirAutorAsync(int id)
{
    var autor = await _context.Autores.FirstOrDefaultAsync(x => x.Id == id);
    if (autor == null) return false;

    _context.Autores.Remove(autor);
    await _context.SaveChangesAsync();
    return true;
}
```

3º Adicionar as Actions na Controller

Agora no arquivo `Controllers/BibliotecaController.cs` colocamos as actions get e post. tanto no `HttpGet` quanto no `HttpPost` nós consultamos o repositório para garantir que a regra seja respeitada.

```
// GET: Biblioteca/DeletarAutor/Id
[HttpGet]
public async Task<IActionResult> DeletarAutor(int id)
{
    var autores = await _autorRepository.BuscarTodosAutoresAsync
();
    var autor = autores.FirstOrDefault(x => x.Id == id);

    if (autor == null) return NotFound();

    // Executa a checagem se há livros vinculados
    bool temLivros = await _autorRepository.PossuiLivrosVinculados
Async(id);

    var viewModel = new DeletarAutorViewModel
    {
        Id = autor.Id,
        Nome = autor.Nome,
        PossuiLivrosVinculados = temLivros
    };

    return View(viewModel);
}

// POST: Biblioteca/DeletarAutor
[HttpPost, ActionName("DeletarAutor")]
public async Task<IActionResult> DeletarAutorConfirmado(int id)
{
    // Proteção extra: Segurança caso o usuário tente forçar a req
uisição POST externamente
    bool temLivros = await _autorRepository.PossuiLivrosVinculados
Async(id);

    if (temLivros)
    {
        // Se houver livros, cancela a operação e joga de volta pa
ra a listagem
        return RedirectToAction("Autor");
    }
}
```

```

    await _autorRepository.ExcluirAutorAsync(id);
    return RedirectToAction("Autor");
}

```

4º Criar a View com Condicional de Alerta

Necessário criar o arquivo `Views/Biblioteca/DeletarAutor.cshtml`

Usamos um bloco `@if` do Razor. Se `PossuiLivrosVinculados` for verdadeiro, exibimos um aviso de bloqueio e escondemos o botão de confirmação. Se for falso, a exclusão prossegue normalmente.

```

@model Biblioteca.Models.DeletarAutorViewModel

@{
    ViewData["Title"] = "Excluir Autor";
}

<div class="container mt-4">
    <h1 class="text-danger">Excluir Autor</h1>
    <hr />

    @if (Model.PossuiLivrosVinculados)
    {
        <div class="alert alert-danger border-danger p-4">
            <h4 class="alert-heading"><i class="bi bi-exclamation-triangle-fill"></i> Operação Bloqueada!</h4>
            <p>Não é possível excluir o autor <strong>@Model.Nome</strong> porque ele possui livros vinculados ao seu cadastro.</p>
            <hr>
            <p class="mb-0 text-muted">Para excluir este autor, você deve primeiro excluir todos os livros associados a ele ou transferi-los para outro autor.</p>
        </div>

        <div class="mt-3">
            <a asp-action="Autor" class="btn btn-secondary">Voltar para Listagem</a>
        </div>
    }
    else

```

```

{
    <h3 class="text-muted">Tem certeza de que deseja excluir p
ermanentemente este autor?</h3>

    <div class="card border-warning mb-4 mt-3">
        <div class="card-header bg-warning text-dark">
            <strong>Confirmação de Dados</strong>
        </div>
        <div class="card-body">
            <dl class="row mb-0">
                <dt class="col-sm-3">Nome do Autor:</dt>
                <dd class="col-sm-9">@Model.Nome</dd>
            </dl>
        </div>
    </div>

    <form asp-action="DeletarAutor" method="post">
        <input type="hidden" asp-for="Id" name="id" />

        <button type="submit" class="btn btn-danger">Sim, Excl
uir</button>
        <a asp-action="Autor" class="btn btn-secondary">Cancel
ar</a>
    </form>
}
</div>

```

5º Adicionar o Botão na Listagem de Autores

Na sua view de listagem de autores insira o botão vermelho de exclusão ao lado do de editar na sua tabela de autores:

```

<td>
    <a asp-action="EditarAutor" asp-route-id="@autor.Id" class="bt
n btn-sm btn-primary">Editar</a>
    <a asp-action="DeletarAutor" asp-route-id="@autor.Id" class="b
tn btn-sm btn-danger">Excluir</a>
</td>

```

🤔 Qualquer dúvida converse com o professor em sala, ou então entre em contato pelo e-mail:

joao.noschang@colaborador.ifmt.edu.br